

**LAPORAN PRAKTIKUM**  
**PRAKTIKUM PENGUJIAN PERANGKAT LUNAK**  
**PERTEMUAN 11**  
**AUTOMASI PENGUJIAN PERANGKAT LUNAK**  
**MENGGUNAKAN CUCUMBER DAN SELENIUM**



**Disusun oleh:**

**Nama** : Januarsyah Akbar  
**NIM** : 24/535846/SV/24314  
**Kelas** : B2  
**Dosen Pengampu** : Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.

**PROGRAM STUDI D-IV**  
**TEKNOLOGI REKAYASA PERANGKAT LUNAK**  
**DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA**  
**SEKOLAH VOKASI**  
**UNIVERSITAS GADJAH MADA**  
**YOGYAKARTA**  
**2026**

# Daftar Isi

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>Daftar Isi</b> . . . . .                                        | <b>ii</b> |
| <b>Tujuan Praktikum</b> . . . . .                                  | <b>1</b>  |
| <b>Langkah Praktikum</b> . . . . .                                 | <b>2</b>  |
| 2.1 Penambahan Dependensi di Awal Langkah . . . . .                | 2         |
| 2.2 Implementasi Page Object Model (POM) . . . . .                 | 3         |
| 2.3 Pembuatan Test Runner . . . . .                                | 5         |
| <b>Hasil dan Pembahasan</b> . . . . .                              | <b>6</b>  |
| 3.1 Latihan: Pengujian Fitur Pencarian (Search Feature) . . . . .  | 6         |
| 3.1.1 Gherkin Feature File - <code>search.feature</code> . . . . . | 6         |
| 3.1.2 Step Definitions - <code>SearchSteps.java</code> . . . . .   | 6         |
| 3.1.3 Hasil Pengujian Fitur Pencarian . . . . .                    | 7         |
| 3.2 Tugas: Pengujian Fitur Login (Login Feature) . . . . .         | 8         |
| 3.2.1 Gherkin Feature File - <code>login.feature</code> . . . . .  | 8         |
| 3.2.2 Step Definitions - <code>LoginSteps.java</code> . . . . .    | 9         |
| 3.2.3 Implementasi Cucumber Hooks . . . . .                        | 10        |
| 3.2.4 Hasil Pengujian Fitur Login . . . . .                        | 11        |
| <b>Kesimpulan</b> . . . . .                                        | <b>12</b> |
| 4.1 Repositori Kode Sumber (Resource) . . . . .                    | 12        |

# Tujuan Praktikum

1. Memahami konsep pengujian otomatis berbasis perilaku (*Behavior Driven Development*) menggunakan Cucumber.
2. Mengonfigurasi dependensi Maven serta mengimplementasikan pola desain *Page Object Model* (POM) dengan Selenium WebDriver.
3. Merancang, mengeksekusi, dan menganalisis pengujian skenario pencarian (Latihan) serta autentikasi masuk akun (Tugas) menggunakan sintaks Gherkin bahasa Indonesia.

# Langkah Praktikum

## 2.1 Penambahan Dependensi di Awal Langkah

Sebelum menulis kode pengujian, dependensi Cucumber dan Selenium dikonfigurasi di dalam file `pom.xml` proyek Maven. Penggunaan *Bill of Materials* (BOM) diterapkan untuk menyelaraskan versi Cucumber secara otomatis tanpa harus mendefinisikan nomor versi pada setiap *sub-dependency*.

File `pom.xml` yang dikonfigurasi berisi kode sebagai berikut:

```
1 <!-- Potongan berkas pom.xml untuk Penambahan Dependensi -->
2 <dependencies>
3   <!-- Selenium Java Driver -->
4   <dependency>
5     <groupId>org.seleniumhq.selenium</groupId>
6     <artifactId>selenium-java</artifactId>
7     <version>4.21.0</version>
8   </dependency>
9   <!-- Safari Driver (Opsional/Pendukung OS Mac) -->
10  <dependency>
11    <groupId>org.seleniumhq.selenium</groupId>
12    <artifactId>selenium-safari-driver</artifactId>
13    <version>4.21.0</version>
14  </dependency>
15  <!-- JUnit 5 API -->
16  <dependency>
17    <groupId>org.junit.jupiter</groupId>
18    <artifactId>junit-jupiter-api</artifactId>
19    <version>5.10.2</version>
20    <scope>test</scope>
21  </dependency>
22  <!-- Cucumber Java Support -->
23  <dependency>
24    <groupId>io.cucumber</groupId>
25    <artifactId>cucumber-java</artifactId>
26    <scope>test</scope>
27  </dependency>
28  <!-- Cucumber JUnit Integration Runner -->
29  <dependency>
30    <groupId>io.cucumber</groupId>
31    <artifactId>cucumber-junit</artifactId>
32    <scope>test</scope>
33  </dependency>
34  <!-- JUnit 4 Engine untuk eksekusi Test Runner standard -->
35  <dependency>
36    <groupId>junit</groupId>
37    <artifactId>junit</artifactId>
38    <version>4.13.2</version>
39    <scope>test</scope>
40  </dependency>
41 </dependencies>
```

Listing .1: Konfigurasi Dependensi Cucumber dan Selenium di `pom.xml`

## 2.2 Implementasi Page Object Model (POM)

Untuk mendukung struktur kode yang bersih, dibuat 5 berkas kelas halaman (*Page Classes*) di dalam paket `pages` pada direktori `src/main/java/pages`:

### 1. `basePage.java`

Merupakan kelas induk (*parent class*) yang mewadahi inisialisasi `WebDriver` dan `WebDriverWait` serta mendefinisikan metode pembantu umum (*generic helper methods*) agar tidak perlu dideklarasikan berulang kali di sub-kelas.

```
1 package pages;
2
3 // [Import statements omitted]
4
5 public class basePage {
6     WebDriver driver;
7     WebDriverWait wait;
8
9     public basePage(WebDriver driver) {
10         this.driver = driver;
11         this.wait = new WebDriverWait(driver, Duration.ofSeconds(5));
12     }
13
14     public WebElement waitUntil(By by) {
15         wait.until(ExpectedConditions.visibilityOfElementLocated(by));
16         return driver.findElement(by);
17     }
18
19     public void click(By by) {
20         waitUntil(by).click();
21     }
22
23     public void sendKeys(By by, String query) {
24         waitUntil(by).sendKeys(query);
25     }
26 }
```

Listing .2: Implementasi Kelas `basePage.java`

### 2. `searchPage.java`

Kelas representasi halaman utama mesin pencari Bing, mewarisi `basePage` dan mendefinisikan pencari lokasi (*locators*) untuk kolom input pencarian dan tombol/form pencarian.

```
1 package pages;
2
3 // [Import statements omitted]
4
5 public class searchPage extends basePage {
6     public searchPage(WebDriver driver) {
7         super(driver);
8     }
9
10     By searchBar = By.id("sb_form_q");
11     By searchForm = By.id("sb_form");
12
13     public void enterQuery(String query) {
14         sendKeys(searchBar, query);
15     }
16
17     public void submitForm() {
18         waitUntil(searchForm).submit();
19     }
20 }
```

Listing .3: Implementasi Kelas `searchPage.java`

### 3. resultPage.java

Kelas representasi halaman hasil pencarian Bing yang bertugas menangkap judul halaman guna proses validasi kecocokan pencarian.

```
1 package pages;
2
3 // [Import statements omitted]
4
5 public class resultPage extends basePage {
6     public resultPage(WebDriver driver) {
7         super(driver);
8     }
9
10    public String getPageTitle() {
11        return driver.getTitle();
12    }
13 }
```

Listing .4: Implementasi Kelas resultPage.java

### 4. loginPage.java

Kelas representasi halaman login situs SauceDemo, mendefinisikan *locators* kolom pengguna, kata sandi, tombol masuk, serta kontainer pesan kesalahan (*error message*).

```
1 package pages;
2
3 // [Import statements omitted]
4
5 public class loginPage extends basePage {
6     public loginPage(WebDriver driver) {
7         super(driver);
8     }
9
10    By usernameField = By.id("user-name");
11    By passwordField = By.id("password");
12    By loginButton = By.id("login-button");
13    By errorMessage = By.cssSelector("[data-test='error']");
14
15    public void enterUsername(String username) {
16        sendKeys(usernameField, username);
17    }
18
19    public void enterPassword(String password) {
20        sendKeys(passwordField, password);
21    }
22
23    public void clickLogin() {
24        click(loginButton);
25    }
26
27    public boolean isErrorMessageDisplayed() {
28        return waitUntil(errorMessage).isDisplayed();
29    }
30 }
```

Listing .5: Implementasi Kelas loginPage.java

## 5. inventoryPage.java

Kelas representasi halaman *inventory* (dasbor) pasca-login di SauceDemo yang memverifikasi keberhasilan masuk lewat kemunculan daftar produk.

```
1 package pages;
2
3 // [Import statements omitted]
4
5 public class inventoryPage extends basePage {
6     public inventoryPage(WebDriver driver) {
7         super(driver);
8     }
9
10    By inventoryList = By.className("inventory_list");
11
12    public boolean isInventoryDisplayed() {
13        return waitUntil(inventoryList).isDisplayed();
14    }
15 }
```

Listing .6: Implementasi Kelas inventoryPage.java

## 2.3 Pembuatan Test Runner

Sebuah kelas bernama `RunCucumberTest.java` dibuat pada paket `runner` (`src/test/java/runner`). Berbeda dengan JUnit 4 tradisional yang memakai `@RunWith(Cucumber.class)`, implementasi ini menggunakan **JUnit Platform Suite Engine** (JUnit 5) agar dapat mengonfigurasi parameter filter tag secara deklaratif via anotasi `@ConfigurationParameter` untuk menyaring skenario pengujian yang akan dijalankan.

```
1 package runner;
2
3 import io.cucumber.junit.platform.engine.Constants;
4 import org.junit.platform.suite.api.ConfigurationParameter;
5 import org.junit.platform.suite.api.IncludeEngines;
6 import org.junit.platform.suite.api.SelectClasspathResource;
7 import org.junit.platform.suite.api.Suite;
8
9 @Suite
10 @IncludeEngines("cucumber")
11 @SelectClasspathResource("features")
12 @ConfigurationParameter(key = Constants.GLUE_PROPERTY_NAME, value = "stepDef")
13 @ConfigurationParameter(key = Constants.PLUGIN_PROPERTY_NAME, value = "pretty, html:target/cucumber-
14 ↪ reports.html")
15 @ConfigurationParameter(key = Constants.FILTER_TAGS_PROPERTY_NAME, value = "@Tugas")
16 public class RunCucumberTest {
17 }
```

Listing .7: Implementasi Kelas RunCucumberTest.java dengan JUnit Platform Suite

# Hasil dan Pembahasan

## 3.1 Latihan: Pengujian Fitur Pencarian (Search Feature)

Pada sesi Latihan, skenario difokuskan pada pengujian otomatis fitur pencarian di mesin pencari Bing menggunakan sintaks Gherkin bahasa Indonesia.

### 3.1.1 Gherkin Feature File - search.feature

Berkas `search.feature` diletakkan di direktori `src/test/resources/features/search.feature`:

```
1 @Latihan
2 Feature: Search functionality on Bing
3
4   Scenario: Search for a keyword on Bing
5     Given aku buka halaman pencarian Bing
6     When aku ngetik kata kunci "Cucumber Java"
7     And aku tekan enter buat nyari
8     Then harusnya muncul hasil yang ada hubungannya sama "Cucumber Java"
```

Listing .1: Skenario Uji search.feature dengan Bahasa Gherkin

### 3.1.2 Step Definitions - SearchSteps.java

Kelas pemetaan langkah fungsional pencarian Bing diimplementasikan di `src/test/java/stepDef/SearchSteps.java` dengan mengacu pada inisialisasi `WebDriver` dari `Cucumber Hooks`:

```
1 package stepDef;
2
3 import io.cucumber.java.en.And;
4 import io.cucumber.java.en.Given;
5 import io.cucumber.java.en.Then;
6 import io.cucumber.java.en.When;
7 import org.junit.Assert;
8 import pages.searchPage;
9 import pages.resultPage;
10
11 public class SearchSteps {
12     searchPage search;
13     resultPage result;
14
15     @Given("aku buka halaman pencarian Bing")
16     public void iAmOnTheBingSearchPage() {
17         Hooks.driver.get("https://www.bing.com");
18         search = new searchPage(Hooks.driver);
19     }
20
21     @When("aku ngetik kata kunci {string}")
```



```

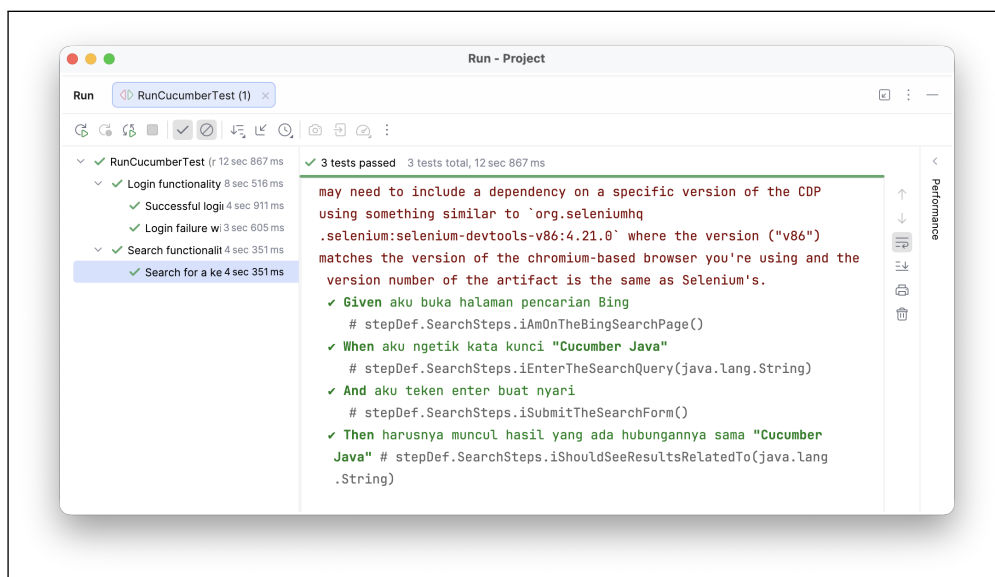
22 public void iEnterTheSearchQuery(String query) {
23     search.enterQuery(query);
24 }
25
26 @And("aku tekan enter buat nyari")
27 public void iSubmitTheSearchForm() {
28     search.submitForm();
29 }
30
31 @Then("harusnya muncul hasil yang ada hubungannya sama {string}")
32 public void iShouldSeeResultsRelatedTo(String query) {
33     result = new resultPage(Hooks.driver);
34     String title = result.getPageTitle();
35     Assert.assertTrue("Judul halaman '" + title + "' gak ada hubungannya sama '" + query + "'",
36         title.toLowerCase().contains(query.toLowerCase()));
37 }
38 }

```

Listing .2: Implementasi Kelas Langkah Uji SearchSteps.java

### 3.1.3 Hasil Pengujian Fitur Pencarian

Eksekusi pengujian dilakukan melalui kelas `RunCucumberTest`. Hasil pengujian fungsionalitas pencarian Bing berjalan sukses dengan indikasi centang hijau dan semua asersi lulus uji (*PASSED*).



Gambar 1: Hasil Eksekusi Pengujian Fitur Pencarian Bing

## 3.2 Tugas: Pengujian Fitur Login (Login Feature)

Pada bagian Tugas Mandiri, skenario difokuskan untuk menguji fitur autentikasi pada platform e-commerce sandbox SauceDemo. Pengujian dirancang mencakup dua kondisi: skenario masuk sukses (positif) menggunakan *Scenario Outline* dengan contoh data, dan skenario masuk gagal (negatif).

### 3.2.1 Gherkin Feature File - login.feature

Berkas login.feature diletakkan di direktori src/test/resources/features/login.feature:

```
1 @Tugas
2 Feature: Login functionality for SauceDemo
3
4 Scenario Outline: Successful login with valid credentials
5     Given aku lagi di halaman login SauceDemo
6     When aku masukan username "<username>" sama password "<password>"
7     And aku klik tombol loginnya
8     Then aku harusnya langsung masuk ke halaman inventory
9
10    Examples:
11        | username      | password      |
12        | standard_user | secret_sauce |
13
14 Scenario: Login failure with invalid credentials
15     Given aku lagi di halaman login SauceDemo
16     When aku masukan username "user_ngasal" sama password "pass_ngasal"
17     And aku klik tombol loginnya
18     Then harusnya muncul pesan error
```

Listing .3: Skenario Uji login.feature dengan Scenario Outline

### 3.2.2 Step Definitions - LoginSteps.java

Kelas implementasi langkah fungsional untuk autentikasi login SauceDemo ditulis pada `src/test/java/stepDef/LoginSteps.java` dengan mengacu pada inisialisasi `WebDriver` dari `Cucumber Hooks`:

```
1 package stepDef;
2
3 import io.cucumber.java.en.And;
4 import io.cucumber.java.en.Given;
5 import io.cucumber.java.en.Then;
6 import io.cucumber.java.en.When;
7 import org.junit.Assert;
8 import pages.loginPage;
9 import pages.inventoryPage;
10
11 public class LoginSteps {
12     loginPage login;
13     inventoryPage inventory;
14
15     @Given("aku lagi di halaman login SauceDemo")
16     public void iAmOnTheSauceDemoLoginPage() {
17         Hooks.driver.get("https://www.saucedemo.com/");
18         login = new loginPage(Hooks.driver);
19     }
20
21     @When("aku masukan username {string} sama password {string}")
22     public void iEnterUsernameAndPassword(String username, String password) {
23         login.enterUsername(username);
24         login.enterPassword(password);
25     }
26
27     @And("aku klik tombol loginnya")
28     public void iClickTheLoginButton() {
29         login.clickLogin();
30     }
31
32     @Then("aku harusnya langsung masuk ke halaman inventory")
33     public void iShouldBeRedirectedToTheInventoryPage() {
34         inventory = new inventoryPage(Hooks.driver);
35         Assert.assertTrue("Halaman inventory gak muncul nih", inventory.isInventoryDisplayed());
36     }
37
38     @Then("harusnya muncul pesan error")
39     public void iShouldSeeAnErrorMessage() {
40         Assert.assertTrue("Pesan error malah gak ada", login.isErrorMessageDisplayed());
41     }
42 }
```

Listing .4: Implementasi Kelas Langkah Uji LoginSteps.java

### 3.2.3 Implementasi Cucumber Hooks

Untuk mengotomatisasi daur hidup `WebDriver` (*browser creation and teardown*), diimplementasikan modul **Cucumber Hooks** (`@Before` dan `@After`). Daurl hidup diatur secara global dalam satu berkas terpisah bernama `Hooks.java` di dalam paket `stepDef`:

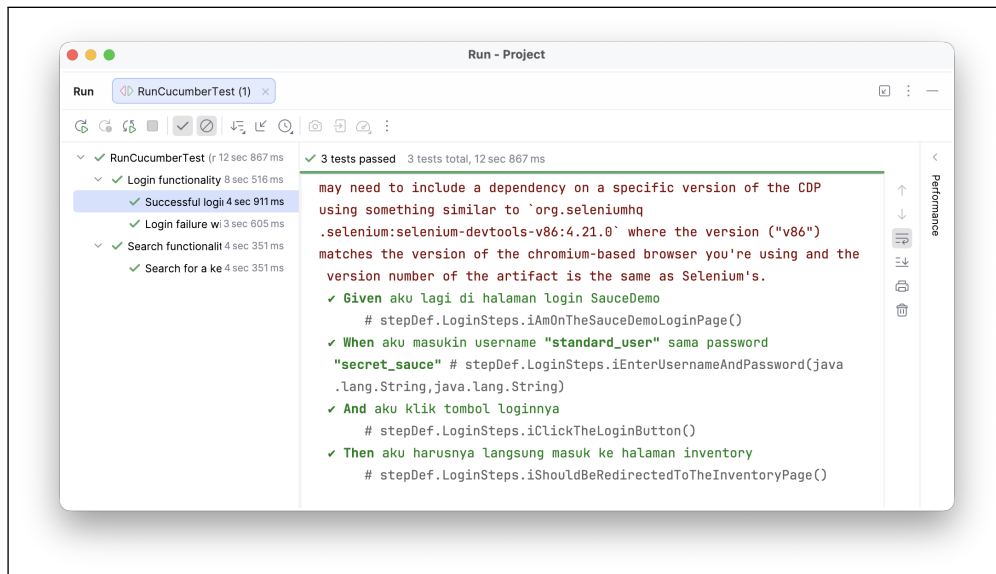
```
1 package stepDef;
2
3 import io.cucumber.java.Before;
4 import io.cucumber.java.After;
5 import org.openqa.selenium.WebDriver;
6 import org.openqa.selenium.chrome.ChromeDriver;
7 import java.time.Duration;
8
9 public class Hooks {
10     public static WebDriver driver;
11
12     @Before
13     public void setUp() {
14         if (driver == null) {
15             driver = new ChromeDriver();
16             driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
17         }
18     }
19
20     @After
21     public void tearDown() {
22         if (driver != null) {
23             driver.quit();
24             driver = null;
25         }
26     }
27 }
```

Listing .5: Implementasi Kelas Hooks.java

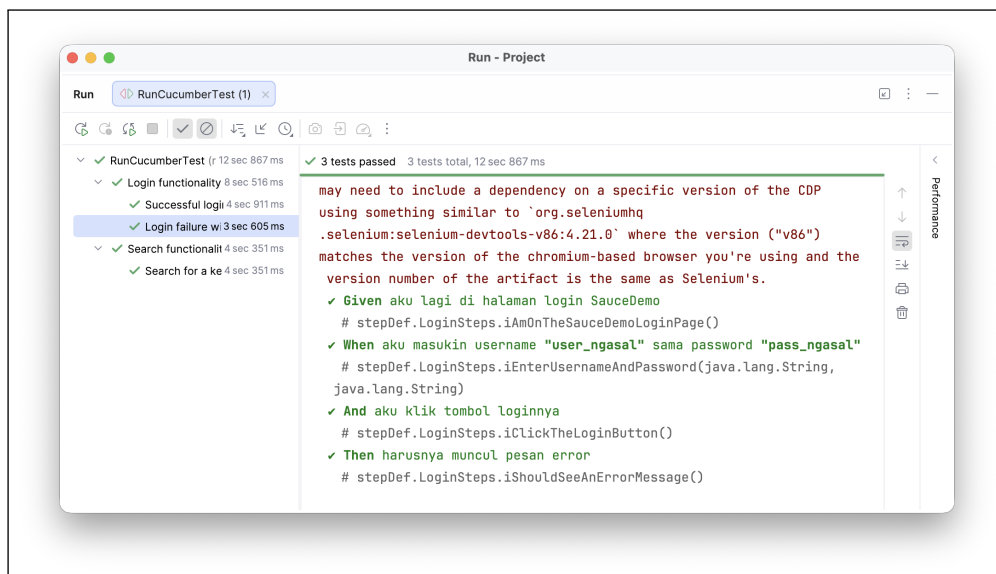
Dengan diimplementasikannya berkas `Hooks.java`, pemanggilan inisialisasi `new ChromeDriver()` dan penutupan browser `driver.quit()` pada masing-masing langkah pengujian fungsional (`LoginSteps.java` dan `SearchSteps.java`) dapat dieliminasi secara utuh. Hal ini mencegah terjadinya redudansi kode serta kebocoran memori (*browser ghost processes*) jika ada asersi uji yang gagal di tengah proses.

### 3.2.4 Hasil Pengujian Fitur Login

Pengujian dijalankan secara otomatis dengan JUnit Runner. Hasil eksekusi menunjukkan kedua skenario autentikasi berhasil diselesaikan tanpa kesalahan (*All Passed*):



Gambar 2: Hasil Eksekusi Skenario Login Berhasil (Positive Case)



Gambar 3: Hasil Eksekusi Skenario Login Gagal (Negative Case)

# Kesimpulan

Praktikum Pertemuan 11 ini berhasil mempraktikkan proses automasi pengujian menggunakan **Cucumber** dan **Selenium WebDriver** dengan pendekatan **Behavior Driven Development** (BDD) dan pola desain **Page Object Model** (POM).

Proyek berhasil dikonfigurasi menggunakan Maven dengan mendefinisikan versi Cucumber 7.34.3 dan Selenium 4.21.0 secara selaras. Implementasi skenario pencarian Bing (Latihan) dan skenario masuk akun SauceDemo (Tugas) yang didefinisikan dalam sintaks Gherkin bahasa Indonesia telah terbukti berjalan dengan sukses, asersi lulus 100%, dan tercatat dalam berkas laporan pengujian otomatis secara komprehensif. Penerapan pola POM terbukti mempermudah perawatan kode uji dengan memisahkan representasi elemen laman dari alur logika skenario pengujian fungsional.

## 4.1 Repositori Kode Sumber (Resource)

Dapatkan full code pada link github di bawah ini:

<https://github.com/januarsyah901/PPPL/tree/5a3220e3abf508bef4469c541536b9ecb77a9eed/Pertemuan%2011>